Next Docs - Full Code Flow Guide

Project explanation, file connections, feature answers, code flow, and used skills for the Next.js collaborative document editor.

Workspace: `C:\Users\DELL\next-doc-editor`
Prepared date: 2026-06-28
Verification: `npm run test` passed, `npm run lint` passed.

1. Project Summary

Next Docs is a local-first collaborative document editor. It allows users to register, log in, create documents, edit rich text, autosave changes, invite collaborators, work offline, resolve conflicts, restore old versions, and see realtime collaborator updates.

The application uses the Next.js App Router for pages and API routes, MongoDB with Mongoose for persistent data, NextAuth credentials login for authentication, Socket.IO for realtime document room updates, Dexie/IndexedDB for offline storage, and Zod for input validation.

2. Technology Stack And Skills Used

Area	Technology / Skill	Where It Appears	Purpose
Frontend	Next.js 16 App Router, React 19, TypeScript	`app/`, `components/`, `hooks/`	Pages, client components, editor UI, dashboard UI, typed application code.
Styling	Tailwind CSS 4	`app/globals.css`, component class names	Responsive layout, cards, forms, rich editor styling, application theme.
Authentication	NextAuth credentials, JWT sessions, bcryptjs	`lib/auth.ts`, `app/api/auth/[...nextauth]/route.ts`	Email/password login and server-side session access.
Database	MongoDB, Mongoose	`lib/mongodb.ts`, `models/`	User, document, collaborator, and version storage.
Validation	Zod	`validators/auth.ts`, `validators/document.ts`	Registration, document, invite, and update input validation.
Realtime	Socket.IO	`server.js`, `hooks/useSocket.ts`	Document rooms, live updates, collaborator presence, room join/leave events.
Offline-first	Dexie, IndexedDB, service worker	`lib/offline-db.ts`, `public/sw.js`	Local document cache, queued saves, offline fallback editor page.
Email	Nodemailer SMTP	`lib/email.ts`	Optional collaborator invitation emails.
Testing	Node test runner, tsx	`tests/`	Unit and integration-style checks for permissions, text parsing, and conflicts.

3. High-Level Architecture

```
Browser UI
  -> Next.js pages in app/
  -> Client components in components/
  -> API routes in app/api/
  -> Session helpers in lib/session.ts
  -> Permission helpers in lib/permissions.ts
  -> Mongoose models in models/
  -> MongoDB

Realtime path:
Browser DocumentEditor -> useSocket -> server.js Socket.IO room -> other browser clients

Offline path:
DocumentEditor -> Dexie IndexedDB queue -> syncQueue -> PATCH /api/documents/[id]
```

4. Folder-Level Responsibility

Folder / File	Responsibility

`app/`	Next.js App Router pages, layouts, protected pages, and API route handlers.
`components/`	Reusable React UI and feature components such as auth form, dashboard, editor, side panels.
`hooks/`	Client hooks for Socket.IO connection and debounced autosave behavior.
`lib/`	Core business helpers: auth, DB connection, sessions, permissions, serializers, conflicts, security, email, offline DB.
`models/`	Mongoose schemas for users, documents, collaborators, and versions.
`validators/`	Zod schemas used by API routes before writing data.
`types/`	Shared TypeScript interfaces and NextAuth session/JWT augmentation.
`utils/`	Small utility helpers such as avatar generation.
`public/`	Service worker and offline fallback HTML page.
`tests/`	Automated tests for important logic.
`server.js`	Custom Next.js HTTP server plus Socket.IO server.
`proxy.ts`	Route guard that redirects unauthenticated users away from protected pages.

5. Runtime Startup Flow

1. `npm run dev` runs `node server.js`.
2. `server.js` creates a custom HTTP server and attaches Next.js request handling.
3. `server.js` also creates a Socket.IO server at `/api/socket`.
4. Browser requests are handled by Next.js pages/API routes.
5. Realtime editor events are handled by Socket.IO rooms in `server.js`.

6. Route And Page Flow

Route	File	Main Logic	Connected Files
`/`	`app/page.tsx`	Landing page with links to register and login.	`components/Navbar.tsx`
`/login`	`app/login/page.tsx`	Shows login form.	`components/AuthForm.tsx`, `components/Navbar.tsx`
`/register`	`app/register/page.tsx`	Shows registration form.	`components/AuthForm.tsx`, `components/Navbar.tsx`
`/dashboard`	`app/dashboard/page.tsx`	Requires session, queries accessible documents, serializes them for client UI.	`lib/session.ts`, `lib/mongodb.ts`, `models/Document.ts`, `lib/permissions.ts`, `lib/serializers.ts`, `components/DashboardClient.tsx`
`/documents/[id]`	`app/documents/[id]/page.tsx`	Requires session, loads accessible document, returns not found if unavailable, passes data to editor.	`components/DocumentEditor.tsx`, `lib/session.ts`, `lib/permissions.ts`, `lib/serializers.ts`
Not found	`app/not-found.tsx`	Friendly fallback when document is missing or access is denied.	`components/Navbar.tsx`

7. API Route Flow

API	Methods	File	Logic
`/api/auth/register`	POST	`app/api/auth/register/route.ts`	Rate limits, reads limited JSON, validates registration, checks duplicate user, hashes password, creates user with avatar.

`/api/auth/[...nextauth]`	GET, POST	`app/api/auth/[...nextauth]/route.ts`	Delegates auth requests to NextAuth using `lib/auth.ts`.
`/api/documents`	GET	`app/api/documents/route.ts`	Requires API user, builds accessible document query, optionally searches by title/content, serializes results.
`/api/documents`	POST	`app/api/documents/route.ts`	Validates title, creates document, adds owner collaborator, creates initial version.
`/api/documents/[id]`	GET	`app/api/documents/[id]/route.ts`	Loads one accessible document and serializes it.
`/api/documents/[id]`	PATCH	`app/api/documents/[id]/route.ts`	Checks edit permission, validates update, sanitizes HTML, checks revision conflict, saves revision, optionally creates version.
`/api/documents/[id]`	DELETE	`app/api/documents/[id]/route.ts`	Only owner can delete. Deletes versions, then deletes document.
`/api/documents/[id]/collaborators`	POST	`app/api/documents/[id]/collaborators/route.ts`	Only owner can invite/update collaborator role. Finds registered user and optionally sends SMTP invite email.
`/api/documents/[id]/collaborators`	DELETE	`app/api/documents/[id]/collaborators/route.ts`	Only owner can remove a non-owner collaborator.
`/api/documents/[id]/versions`	GET	`app/api/documents/[id]/versions/route.ts`	Lists latest 50 versions for an accessible document.

`/api/documents/[id]/versions/[versionId]/restore`	POST	`app/api/documents/[id]/versions/[versionId]/restore/route.ts`	Editor/owner can restore old version. Creates a new version labelled restored.
`/api/test-db`	GET	`app/api/test-db/route.ts`	Simple MongoDB connection health check.

8. File Connection Map

File	Connects To	Reason
`app/layout.tsx`	`app/globals.css`, `components/Providers.tsx`, `components/Footer.tsx`, `components/ServiceWorkerRegister.tsx`	Wraps all pages with global CSS, auth provider, toast provider, service worker registration, and footer.
`components/Providers.tsx`	`next-auth/react`, `react-hot-toast`	Makes session state and toast notifications available to client components.
`proxy.ts`	`next-auth/jwt`	Checks token before protected page access and redirects login/register users correctly.
`components/AuthForm.tsx`	`/api/auth/register`, `signIn("credentials")`	Registers new users, then logs in through NextAuth credentials.
`lib/auth.ts`	`lib/mongodb.ts`, `models/User.ts`, `bcryptjs`	Credentials authorize function finds user and compares password hash.
`lib/session.ts`	`lib/auth.ts`	Gets server session and exposes `requireUser` / `requireApiUser`.
`components/DashboardClient.tsx`	`/api/documents`, `/api/documents/[id]`, `lib/client-api.ts`, `lib/text.ts`, `components/ConfirmDialog.tsx`	Creates, searches, opens, and deletes documents in the browser.
`components/DocumentEditor.tsx`	`hooks/useDebouncedCallback.ts`, `hooks/useSocket.ts`, `lib/offline-db.ts`, `lib/client-api.ts`, `lib/conflicts.ts`, side panels	Main feature component for editing, autosave, manual save, offline queue, realtime updates, and conflicts.
`components/EditorTools.tsx`	`lib/text.ts`, parent callbacks from `DocumentEditor`	Displays formatting buttons, stats, preview, and find/replace UI.
`components/CollaboratorsPanel.tsx`	`/api/documents/[id]/collaborators`, `lib/client-api.ts`, `components/ConfirmDialog.tsx`	Invites or removes collaborators and updates collaborator list in editor state.
`components/VersionHistory.tsx`	`/api/documents/[id]/versions`, `/restore`, `lib/client-api.ts`, `components/ConfirmDialog.tsx`	Loads version list and restores selected versions.
`hooks/useSocket.ts`	`server.js`	Connects browser to Socket.IO endpoint and exposes update/presence listeners.
`server.js`	`hooks/useSocket.ts`, browser clients	Runs realtime rooms, validates socket room/payload/rate limits, broadcasts updates and presence.
`lib/permissions.ts`	`models/Document.ts`, `lib/mongodb.ts`, `types/index.ts`	Central document access and role logic used by pages and APIs.
`lib/serializers.ts`	`lib/permissions.ts`	Converts Mongoose documents into safe JSON shape for clients.
`lib/security.ts`	API routes	Shared rate limiting, body size limiting, JSON reading, and HTML sanitization.
`public/sw.js`	`components/ServiceWorkerRegister.tsx`, `public/offline-document.html`	Caches offline fallback page for document routes when navigation fails.

9. Database Model Logic

`models/User.ts`

Stores user name, unique lowercase email, hashed password, avatar URL, last login field, and timestamps. Used by registration, login, and collaborator invitation.

`models/Document.ts`

Stores title, HTML content, owner user reference, collaborator subdocuments, last editor, last saved time, revision number, and timestamps. Collaborators have a user reference, role of `OWNER`, `EDITOR`, or `VIEWER`, and invited time.

`models/Version.ts`

Stores document reference, title snapshot, content snapshot, label, creator reference, and timestamps. Used for initial document version, autosave snapshots, manual saves, and restore history.

10. Authentication Flow

```
Register:
AuthForm -> POST /api/auth/register
  -> enforceRateLimit + readLimitedJson
  -> RegisterSchema
  -> connectDB
  -> User.findOne duplicate check
  -> hashPassword
  -> User.create
  -> AuthForm calls signIn("credentials")

Login:
AuthForm -> signIn("credentials")
  -> /api/auth/[...nextauth]
  -> lib/auth.ts authorize()
  -> connectDB -> User.findOne -> bcrypt.compare
  -> JWT callback stores token.id
  -> session callback exposes session.user.id
```

Protected pages are guarded twice: `proxy.ts` redirects unauthenticated users at the route level, and server pages call `requireUser()` before reading private data. API routes call `requireApiUser()` and return `401 Unauthorized` when no session exists.

11. Dashboard Flow

```
/dashboard page
  -> requireUser()
  -> connectDB()
  -> Document.find(collaboratorQuery(user.id))
  -> populate owner/collaborators/lastEditedBy
  -> serializeDocument(document, user.id)
  -> DashboardClient

DashboardClient actions:
  create -> POST /api/documents -> router.push(/documents/[id])
  search -> local title/plain-text filtering
  delete -> DELETE /api/documents/[id] with optimistic UI rollback on failure
```

12. Editor Flow

```
/documents/[id] page
  -> requireUser()
  -> getAccessibleDocument(id, user.id)
  -> populate related users
  -> serializeDocument()
  -> DocumentEditor

DocumentEditor:
  state = title/content/document/saveStatus/online/conflict/presence
  title input -> debouncedSave
  rich editor input -> syncRichEditor -> socket live emit + debouncedSave
  manual save -> saveNow(..., createVersion=true)
  side panels -> collaborators and versions
  canEdit = OWNER or EDITOR; VIEWER cannot edit
```

13. Save, Autosave, And Version Flow

1. User types in the contenteditable editor.
2. `syncRichEditor()` reads `editorRef.current.innerHTML`.
3. Local state is updated and Socket.IO broadcasts a live update.
4. `useDebouncedCallback` delays the API save by 900ms.
5. `saveNow()` caches the document in IndexedDB before trying the network.
6. If online, it calls `PATCH /api/documents/[id]`.
7. The API validates, sanitizes, checks permission and revision, increments revision, and saves.
8. A version is created for first save, every third saved change, or explicit manual save.

14. Realtime Collaboration Flow

```
Client A opens document
  -> useSocket connects to /api/socket
  -> emits document:join with documentId and user
  -> server.js joins room and broadcasts presence

Client A edits
  -> DocumentEditor emits document:update
  -> server.js validates room, rate, payload size, and sanitizes content
  -> server broadcasts update to other clients in same document room
  -> Client B receives update, refreshes title/content/revision, shows toast
```

The realtime layer is for broadcasting live changes and presence. Database persistence still goes through the normal authenticated API routes, which keeps permission checks and validation centralized.

15. Offline And Sync Flow

```
Online editor:
  -> DocumentEditor caches latest document in Dexie documents table

If browser is offline:
  -> saveNow detects !navigator.onLine
  -> enqueueSave stores operation in Dexie queue table
  -> UI shows queued/offline status

When browser returns online:
  -> online event calls syncQueue()
  -> queued saves PATCH /api/documents/[id]
  -> success deletes queue item
  -> conflict attempts merge with mergeDocumentContent()
  -> retry delay uses exponential backoff up to 60s
```

The service worker in `public/sw.js` provides a fallback page for `/documents/[id]` navigation when the network fails. The fallback page reads and writes the same IndexedDB database, so the user can still queue a save outside the full React app shell.

16. Conflict Resolution Flow

```
Client sends baseRevision
  -> API compares baseRevision with current document.revision
  -> if baseRevision is stale and force is false:
       return 409 conflict with serverDocument and clientDocument

User choices in DocumentEditor:
  Keep server version -> replace local state with server document
  Overwrite with mine -> save client document with force=true
  Merge both -> mergeDocumentContent(server, client, base) then save with force=true
```

Conflict helpers live in `lib/conflicts.ts`. The merge function handles easy cases first: if only one side changed from the base, it keeps that side. If both changed, it creates marked server/local sections so no content silently disappears.

17. Role And Permission Rules

Role	Can View	Can Edit	Can Manage Collaborators	Can Delete
`OWNER`	Yes	Yes	Yes	Yes
`EDITOR`	Yes	Yes	No	No
`VIEWER`	Yes	No	No	No

These rules are centralized in `lib/permissions.ts`. UI components also respect them, but the important enforcement is server-side in page loaders and API routes.

18. Security And Data Safety

- `proxy.ts` blocks protected pages for users without JWT token.
- `requireUser()` and `requireApiUser()` protect server reads and API writes.

- Zod schemas validate auth, document, update, invite, and remove collaborator payloads.
- `readLimitedJson()` rejects oversized request bodies.
- `enforceRateLimit()` applies in-memory rate limits per endpoint scope.
- `sanitizeDocumentHtml()` strips risky tags, event handlers, inline styles, and dangerous URLs.
- Socket payloads are size-limited, rate-limited, room-checked, and sanitized.
- Password storage uses bcrypt hashes, not plain text.

19. Environment Variables

Variable	Used By	Purpose
`MONGODB_URI`	`lib/mongodb.ts`	MongoDB connection string.
`AUTH_SECRET`	`lib/auth.ts`, `proxy.ts`	JWT/session signing secret.
`NEXT_PUBLIC_APP_URL` / `NEXTAUTH_URL`	`server.js`, collaborator API	Socket CORS origin and invite document links.
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM`	`lib/email.ts`	Optional SMTP settings for invite emails.
`NEXT_PUBLIC_PROFILE_NAME`, `NEXT_PUBLIC_PROFILE_GITHUB`, `NEXT_PUBLIC_PROFILE_LINKEDIN`	`components/Footer.tsx`	Footer profile display.

20. Feature Answer Guide

> **Short project pitch:** I built a local-first collaborative document editor with authentication, document CRUD, rich-text editing, debounced autosave, realtime Socket.IO updates, collaborator roles, offline IndexedDB queueing, conflict handling, and version restore.

Feature	How To Explain It	Main Files
Authentication	Users register with email/password. Passwords are hashed with bcrypt. NextAuth credentials creates a JWT session and protected routes require that session.	`AuthForm.tsx`, `register/route.ts`, `lib/auth.ts`, `lib/session.ts`, `proxy.ts`
Document CRUD	Dashboard loads only documents the user owns or collaborates on. Create and delete go through protected API routes.	`DashboardClient.tsx`, `app/dashboard/page.tsx`, `app/api/documents/route.ts`, `app/api/documents/[id]/route.ts`
Rich editor	The editor uses a contenteditable area and applies formatting commands for bold, italic, headings, lists, quotes, code, and links. It also has stats, preview, and find/replace.	`DocumentEditor.tsx`, `EditorTools.tsx`, `app/globals.css`
Autosave	Changes are debounced for 900ms before calling the PATCH document API. Manual save creates an explicit version snapshot.	`DocumentEditor.tsx`, `useDebouncedCallback.ts`, `app/api/documents/[id]/route.ts`
Realtime collaboration	Each document has a Socket.IO room. When one user edits, other users in the same room receive updates and presence events.	`server.js`, `useSocket.ts`, `DocumentEditor.tsx`
Offline support	Every document is cached in IndexedDB. If the browser is offline, saves are queued and synced when online again. A service worker provides a fallback offline document page.	`lib/offline-db.ts`, `DocumentEditor.tsx`, `public/sw.js`, `public/offline-document.html`
Conflict handling	Every save includes a base revision. If the server has a newer revision, the API returns a conflict. The user can keep server, overwrite, or merge both.	`lib/conflicts.ts`, `app/api/documents/[id]/route.ts`, `DocumentEditor.tsx`
Collaborators	Owners can invite registered users as editor or viewer. Roles are enforced both in UI and server APIs. SMTP can send invite emails.	`CollaboratorsPanel.tsx`, `collaborators/route.ts`, `lib/permissions.ts`, `lib/email.ts`
Version history	The app stores snapshots on create, autosave checkpoints, manual saves, and restores. Editors can restore a prior version and the restore itself becomes a new version.	`VersionHistory.tsx`, `models/Version.ts`, `versions/route.ts`, `restore/route.ts`

21. Common Interview / Viva Answers

What is the main idea of this project?

It is a collaborative document editor built with Next.js. It focuses on practical document workflows: secure login, document CRUD, rich editing, collaborator permissions, live updates, offline edits, conflict resolution, and version restore.

How did you protect private documents?

Protected pages are redirected by `proxy.ts`. Server pages and APIs read the session with `lib/session.ts`. Document access uses `collaboratorQuery()` and `getAccessibleDocument()`, so a user only sees documents where they are owner or collaborator.

How does offline editing work?

The editor caches every document in IndexedDB using Dexie. If the browser is offline or a network request fails, the app stores a queue item with the title, content, base revision, and base content. When the browser is online again, `syncQueue()` retries those saves.

How are conflicts handled?

Each document has a revision number. The client sends its base revision with each save. If the server revision is newer, the API returns a 409 conflict with both server and client versions. The user can keep server, overwrite server, or merge both using `mergeDocumentContent()`.

Why use Socket.IO if saves already go to API routes?

Socket.IO is used for realtime communication and presence. API routes are still responsible for persistence, validation, authorization, sanitization, versioning, and conflict checks. This separates live UI feedback from trusted database writes.

What security measures are included?

The project uses hashed passwords, route/session guards, role-based document permissions, Zod validation, body size limits, rate limiting, HTML sanitization, and socket payload checks.

22. Test Coverage

Test File	What It Checks
`tests/unit/permissions.test.ts`	`getUserRole()` detects owner and collaborator roles from populated objects.
`tests/unit/text.test.ts`	`htmlToPlainText()` strips tags, removes scripts, and decodes common HTML entities.
`tests/integration/conflicts.test.ts`	Revision conflict detection and merge behavior for server/client/base content.

23. Quick Demo Script

1. Run `npm run dev` and open `http://localhost:3000`.
2. Register a new account and show that it redirects to the dashboard.
3. Create a document and type content. Explain debounced autosave and manual save.
4. Open another browser/session with another registered user, invite them, and show role behavior.
5. Open the same document in two sessions and show realtime updates/presence.
6. Disconnect network, edit content, reconnect, and show queued sync.
7. Create a manual save, open version history, and restore a previous version.